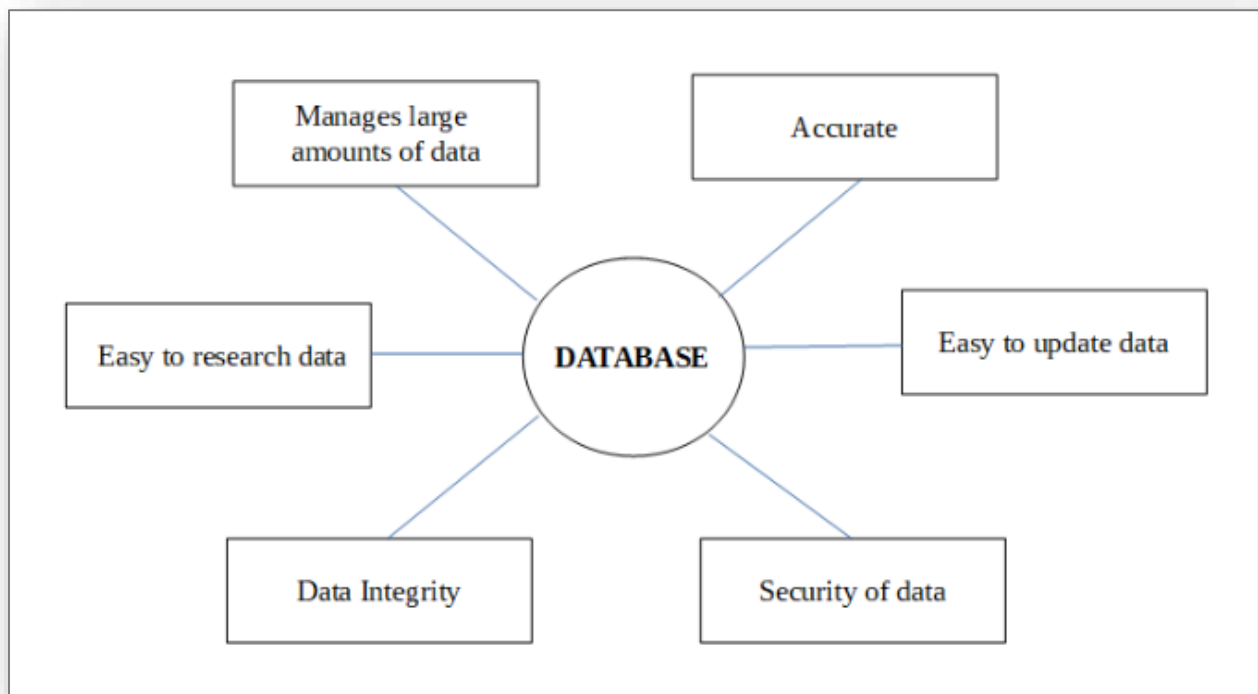


Q.1) What is database?

A database is an organized collection of data, typically stored electronically in a computer system, that can be easily accessed, managed, and updated. Databases are controlled by a database management system (DBMS) and often use structured query language (SQL) to interact with the data.

Q.2) Why database ?

Databases are used to store, manage, and retrieve large amounts of data in an organized and efficient way, enabling businesses and applications to operate by making data accessible to multiple users simultaneously, ensuring data integrity, and providing security. They are crucial for tasks like making informed decisions, automating processes, and powering everything from financial transactions to AI models.



Q.3) Data Level

The three levels of abstraction in a database system, often referred to as the three-level DBMS architecture, are designed to provide data independence and allow different users and applications to interact with the database at varying levels of detail.

Relationship with Database, DBMS, and Application:

➤ *Database:*

The actual collection of stored data, structured according to the conceptual and internal schemas.

➤ *DBMS (Database Management System):*

The software that manages the database and provides the interfaces and tools to interact with it at all three levels. It handles the mapping between the different levels (e.g., translating a user's request from the external view into operations on the internal storage).

➤ *Application:*

Software programs that interact with the database, typically through the external level (views) provided by the DBMS, to perform specific tasks for end-users. Applications rely on the DBMS to handle data storage and retrieval, allowing developers to focus on application logic rather than low-level data management.

//Alternative

1. Database

A **database** is an organized collection of data that is stored and managed so that it can be easily accessed, updated, and retrieved.

Examples: Student information system, banking records, online shopping data.

Key characteristics:

- Organized and structured
- Stores large amounts of data
- Reduces redundancy
- Ensures data integrity and consistency

2. DBMS (Database Management System)

A **DBMS** is software that allows users to create, store, manage, and retrieve data from a database.

Examples: MySQL, Oracle, SQL Server, PostgreSQL.

Functions of DBMS:

- Data storage, retrieval, and update
- Security and authorization
- Backup and recovery
- Data integrity & consistency
- Concurrency control

3. Applications (Database Applications)

Database applications are software programs that interact with the DBMS to perform tasks such as inserting, updating, deleting, and viewing data.

Examples:

- Library management system
- Banking apps
- E-commerce apps
- Hospital management system

These applications send SQL queries to the DBMS, and the DBMS processes them and returns results from the database.

Q.4) Contents of database

A database is a self-describing collection of integrated records that stores various components to manage data efficiently. These components include user data, metadata, application metadata, indexes, and other overhead data.

➤ *User Data*

User data refers to the actual raw information that is directly relevant to the end-users and the primary function of the application.

Personal Information: Personally identifiable information (PII) such as names, addresses, email addresses, and phone numbers.

Behavioral Data: Information gathered from user actions, such as browsing history, purchase history, search history, app interactions, and location data.

User-Generated Content: Data that users knowingly create and share, such as photos, videos, posts, reviews, and messages.

Preferences and Settings: Information on user preferences, requested payments, privacy levels, and application-specific settings that allow for a personalized experience.

Financial and Health Data: Sensitive information like payment details, credit scores, health records, and fitness information, which often require specific security measures.

➤ *Metadata*

Metadata is "data about data" and provides information about the structure, content, and context of the database itself. It is stored in a data dictionary or system catalog.

Schema and Structure: Definitions of database objects, including table names, column names, data types, constraints (e.g., primary keys, foreign keys), and the relationships between tables.

Technical Information: Details about how data is physically stored, such as file formats, storage size, index information, and system parameters.

Administrative Information: Data related to the management and governance of the database, including creation dates, author information, modification times, access rights, and data lineage (history of data transformations).

Operational Information: Data about the current status of data, such as whether data is active, archived, or purged, and information used by managers for daily tasks.

➤ ***Application Metadata***

Application metadata is specific to the software application that interacts with the database. It helps in customizing the application's functionality and user experience.

User Interface Settings: Information on user preferences for the application's appearance or functionality (e.g., dark mode, notification settings).

Configuration Details: Application-specific settings, such as server IP addresses, scheduling information for batch jobs, or default options, that might change often.

App Store Optimization (ASO) data: For mobile apps, this includes the app title, description, keywords, category, version number, and screenshots, all of which help with app discovery and user understanding.

➤ ***Index and Other Overhead Data***

This data is used by the Database Management System (DBMS) primarily for performance optimization, maintenance, and reliable operation. This data consumes additional storage space but significantly speeds up operations.

Indexes: Data structures (like B-trees or hash tables) that store sorted key values and pointers to the actual data rows, enabling faster data retrieval by avoiding full table scans.

Transaction Logs: Records of all database transactions (inserts, updates, deletes). These logs are crucial for ensuring data consistency and for database recovery in case of a system failure (ACID properties).

System Catalogs/Data Dictionary: The central repository for all metadata, used by the DBMS to verify user requests and optimize queries.

Statistics: Data collected by the DBMS about data distribution and usage patterns, which the query optimizer uses to devise the most efficient query execution plans.

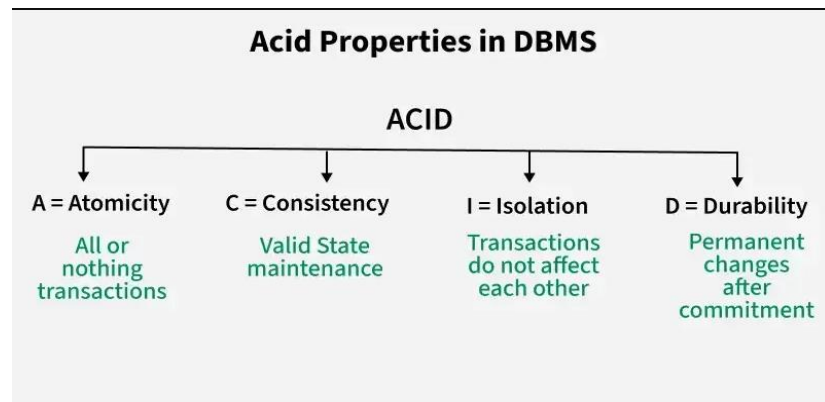
Security and Tracking Data: Information related to user access, security permissions, and audit trails to track who accessed what data and when.

Q.5) 3 Acronyms of database?

Here are the acronyms for ACID, SQL, and CRUD in the context of databases:

➤ **ACID:**

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

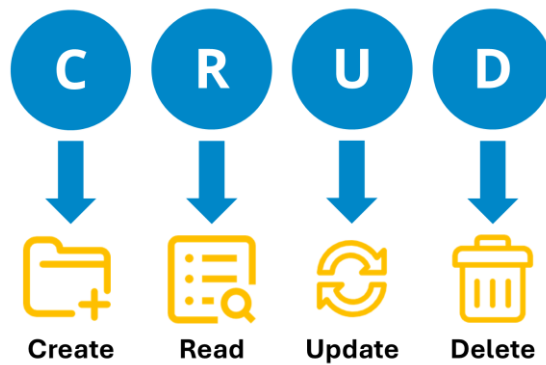


➤ **SQL:**

- **S**tructured
- **Q**uery
- **L**anguage

SQL databases or relational databases are widely used for storing, managing and organizing structured data in a tabular format. These databases store data in tables consisting of rows and columns. SQL is the standard programming language used to interact with these databases. It enables users to create, update, retrieve and manage data effectively.

- **CRUD:**
- Create
 - Read
 - Update
 - Delete

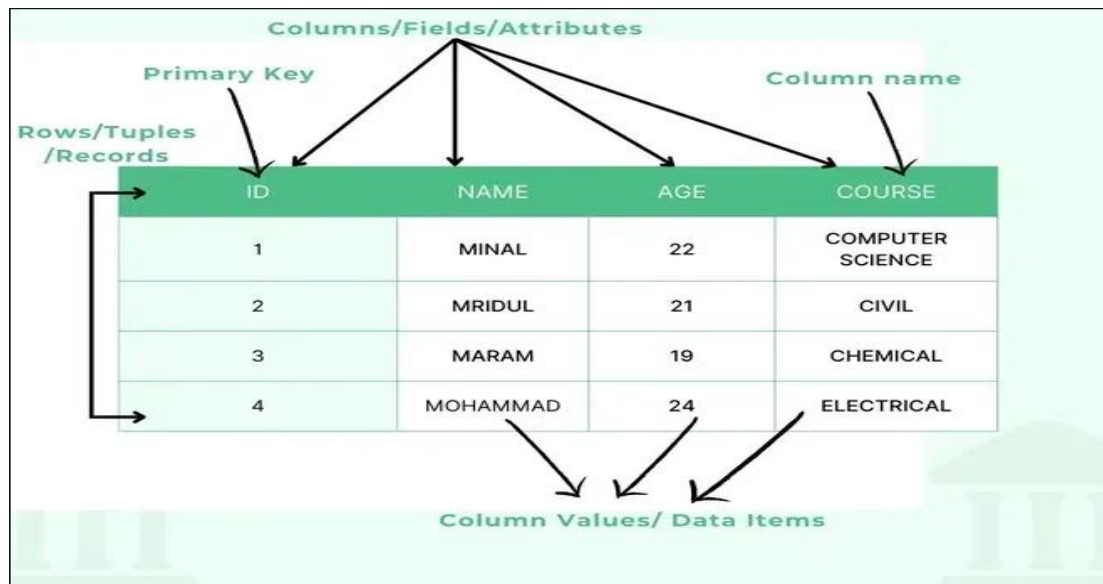


Q.6) Relational Database

RDBMS stands for Relational Database Management Systems. It is a program that allows us to create, delete, and update a relational database. A Relational Database is a database system that stores and retrieves data in a tabular format organized in the form of rows and columns. It is a smaller subset of DBMS which was designed by E.F Codd in the 1970s. The major DBMSs like SQL, My-SQL, and ORACLE are all based on the principles of relational DBMS.

Relational DBMS owes its foundation to the fact that the values of each table are related to others. It has the capability to handle larger magnitudes of data and simulate queries easily.

- **Entity Integrity:** No two records of the database table can be completely duplicate.
- **Referential Integrity:** Only the rows of those tables can be deleted which are not used by other tables. Otherwise, it may lead to data inconsistency.
- **User-defined Integrity:** Rules defined by the users based on confidentiality and access.
- **Domain integrity:** The columns of the database tables are enclosed within some structured limits, based on default values, type of data or ranges.



Q.7) Entity and properties of entity

An entity in DBMS represents a real-world object or concept that can be uniquely identified and stored in a database. It forms the foundation of data modeling in an Entity-Relationship (ER) model.

Example: A student, an employee, or bank a/c, etc. all are entities.

Entity properties are descriptive details (attributes like name, ID, status, color) that define an entity (a real-world object or concept like a user, car, or product) in a system, allowing for data organization, filtering, and unique identification, with properties varying in type (text, number, boolean) and can be indexed for faster searches, forming core data points in databases and data models.

Key Properties & Characteristics

Name/Code: A clear, meaningful label for humans and a technical identifier for systems.

Data Types: Can be numeric (INTEGER, REAL), text (string), boolean (True/False), or structured (structs, slices).

Cardinality: A single property can hold multiple values (e.g., multiple phone numbers for one user).

Indexing: Can be indexed for efficient querying (ordering, filtering) or left unindexed.

Behavioral: Properties can control editing permissions (AllowEdit, ReadOnly) or inherit from underlying data.

Examples in Context:

User Entity: Properties like UserID , HomeCountry , SubscriptionStatus.

Car Entity: Properties like Registration_No, Model, Color.

Product Entity: Properties like Price, SKU, Name.

Why They Matter

Description: Attributes provide meaning to an entity (e.g., a car's color).

Identification: Unique properties (like an account number) identify specific entities.

Organization: Grouping and filtering results in applications.

Data Integrity: Enforcing rules on data types and values.

Q.8) Introduction to keys of Database

Primary Key: A primary key is a type of key that contains a column or set of columns in a table that uniquely identifies each row in the table. The primary key is an attribute or a set of attributes that help to uniquely identify the tuples(records) in the relational table.

Surrogate Key: A manufactured unique identifier created in each database table record. When a natural primary key is not available, this value, which is usually an integer or a system-generated number, is utilized.

Foreign Key: A foreign key is an attribute of a relationship or group of attributes that could serve as the primary key of another relationship to which it is connected through a relationship.

Q.9) Normalization

Ans: Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).

Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.

In the relational model, there exist standard methods to quantify how efficient a database is. These methods are called normal forms and there are algorithms to convert a given database into normal forms.

Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment, Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing

After Normalization

Employee

Emp_ID	Emp_Name	Dept_ID
101	Nick Wise	D1
102	John Cader	D2
103	Lily Case	D1
104	Ford Dawid	D3

Department

Dept_ID	Department	Dept_Location
D1	HR	London
D2	Finance	Australia
D3	IT	Chicago

Employee_Skills

Emp_ID	Emp_Skills
101	Recruitment
101	Payroll
102	Budgeting
103	Recruitment
104	Programming
104	Testing

Problems in the Employee_Department Relation

1. Insertion Anomaly:

- If a new department is created but no employee is assigned to it yet, we cannot store its location because we need an employee record to insert.

2. Update Anomaly:

- If the location of the HR department changes, we must update it in multiple rows (for both Nick Wise and Lily Case). If one row is missed, the data becomes inconsistent.

3. Deletion Anomaly:

- If all employees in the IT department leave, we lose the department information, including its location.

4. Data Redundancy:

- The department location is repeated for every employee in the same department.

Features of Database Normalization

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database. Data redundancy refers to the repetition of data in different parts of the database. Normalization helps in reducing or eliminating this redundancy, which can improve the efficiency and consistency of the database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate. By eliminating redundancy, normalization helps in preventing inconsistencies and contradictions that can arise due to different versions of the same data.
- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database. By breaking down a complex data structure into simpler tables, normalization makes it easier to manage the data, update it, and retrieve it.
- **Improved Database Design:** Normalization helps in improving the overall design of the database. By organizing the data in a structured and systematic way, normalization makes it easier to design and maintain the database. It also makes the database more flexible and adaptable to changing business needs.
- **Avoiding Update Anomalies:** Normalization helps in avoiding update anomalies, which can occur when updating a single record in a table affects multiple records in other tables. Normalization ensures that each table contains only one type of data and that the relationships between the tables are clearly defined, which helps in avoiding such anomalies.

- **Standardization:** Normalization helps in standardizing the data in the database. By organizing the data into tables and defining relationships between them, normalization helps in ensuring that the data is stored in a consistent and uniform manner.

Advantages of Normalization

- Normalization eliminates data redundancy and ensures that each piece of data is stored in only one place, reducing the risk of data inconsistency and making it easier to maintain data accuracy.
- By breaking down data into smaller, more specific tables, normalization helps ensure that each table stores only relevant data, which improves the overall data integrity of the database.
- Normalization simplifies the process of updating data, as it only needs to be changed in one place rather than in multiple places throughout the database.
- Normalization enables users to query the database using a variety of different criteria, as the data is organized into smaller, more specific tables that can be joined together as needed.
- Normalization can help ensure that data is consistent across different applications that use the same database, making it easier to integrate different applications and ensuring that all users have access to accurate and consistent data.

Disadvantages of Normalization

- Normalization can result in increased performance overhead due to the need for additional join operations and the potential for slower query execution times.
- Normalization can result in the loss of data context, as data may be split across multiple tables and require additional joins to retrieve.
- Proper implementation of normalization requires expert knowledge of database design and the normalization process.
- Normalization can increase the complexity of a database design, especially if the data model is not well understood or if the normalization process is not carried out correctly.

PRACTICE QUESTIONS

Batch-45 : d)

At the DIU Library, students often borrow books through the online system.

When a student requests a book, the system updates both the borrowing record and the book's availability at the same time, ensuring the process is either fully completed or not done at all. If a failure occurs midway, all changes are automatically reversed, keeping the data accurate. Even during peak times, when many students request the same textbook, each transaction is handled independently, preventing conflicts. Once a borrowing is confirmed, the record is stored permanently, unaffected by power outages or crashes, ensuring the library's database remains consistent and reliable.

Determine the database transaction properties shown in the scenario and demonstrate their role in handling the process of DIU library.

Ans: Atomicity, Consistency, Isolation, and Durability (ACID properties)

The scenario describes how the DIU library's online system adheres to the ACID properties of database transactions to ensure data integrity and reliability.

- **Atomicity:** The text states the process is "either fully completed or not done at all" and that "all changes are automatically reversed" if a failure occurs midway. This ensures that a book borrowing transaction is treated as a single, indivisible unit of work. The role is to keep the data accurate by preventing partial updates (e.g., a book being marked as borrowed but not associated with a student record).
- **Consistency:** The system "updates both the borrowing record and the book's availability at the same time, ensuring the process is either fully completed or not done at all," which keeps the data accurate. The role is to ensure that the database moves from one valid state to another after a transaction.
- **Isolation:** "Each transaction is handled independently, preventing conflicts" even when "many students request the same textbook". The role is to ensure that concurrent transactions do not interfere with each other, so one student's transaction doesn't see the partial results of another, preventing issues like double-booking a single available book.

- **Durability:** "Once a borrowing is confirmed, the record is stored permanently, unaffected by power outages or crashes". The role is to ensure that once a transaction has been committed, the changes remain permanent and are not lost due to system failures.

Batch-44

Ques. ABC Institute maintains all student, course, and instructor information in a single table. This leads to repeated data, inconsistencies in course names, and difficulty updating instructor details. Managing records has become inefficient and error-prone. The institute now seeks a better way to organize the data to reduce duplication, ensure accuracy, and make updates easier across the system. Determine a technique that they can organize their data according to DBMS..

Answer: The problem described by ABC Institute is a classic database design issue, and in DBMS the correct technique to solve it is called Database Normalization .

Normalization is a technique in DBMS used to organize data efficiently by:

1. Reducing data duplication
2. Eliminating inconsistencies
3. Improving data integrity
4. Making updates easier and safer

Instead of keeping all student, course, and instructor information in one large table, normalization divides the data into multiple related tables.

ABC Institute currently faces:

1. Repeated student, course, and instructor data
2. Inconsistent course names
3. Difficulty updating instructor details
4. High chance of update, insert, and delete anomalies

Normalization directly addresses all these problems.

1. Using normalization, the data can be organized like this:
2. Student Table → StudentID, Name, Email, Department
3. Course Table → CourseID, CourseName, Credits
4. Instructor Table → InstructorID, InstructorName, Contact

5. Enrollment Table → StudentID, CourseID, InstructorID

Each table stores unique data only, and relationships are maintained using primary keys and foreign keys.

Normal Forms Used

1. 1NF (First Normal Form) → Removes repeating groups
2. 2NF (Second Normal Form) → Removes partial dependency
3. 3NF (Third Normal Form) → Removes transitive dependency
4. Most systems aim for 3NF, which is efficient and practical.

Batch-43
Question)

c)

Table: Student					Table: Course		
StudentID	Name	Age	CGPA	CourseID	CourseID	CourseName	Credit
025	Alice	22	3.50	101	101	Mathematics	3
059	Bob	23	3.65	102	102	Physics	1
009	Charlie	22	3.45	103	103	English	2
095	Mark	24	3.85	101	105	Programming	4

In the context of databases, two of the most significant keys are present. These keys are essential for ensuring data integrity and establishing relationships between tables. Detect the two keys from the above tables and demonstrate the difference between these two keys.

Answer: Primary Key and Foreign Key

Primary Key: A primary key is a unique identifier assigned to each record within a database table.

1. A primary key is an attribute (or column) or a set of attributes that uniquely identifies each row (or record) in the table.
2. Since the primary key is used to identify the tuples of a relation none of its attribute values can be null.
3. In many relational DBMSs, when you declare a primary key constraint, the system will create a unique index on that key.

Foreign key :Foreign keys are a set of constraints in DBMS that establish relationships between tables and also ensure consistency and integrity of data.

1. A foreign key is applied to a column (or attribute) of one table which references the primary key of a column in another table.
2. It act as a cross-reference between two tables. It helps to maintain data integrity which is known as "Referential Integrity Constraints".
3. A foreign key ensures that the data in one table corresponds to a valid entry (or entries) in another table.
4. A foreign key can refer to a column in the same table, which is called self-referencing. This is often used to show relationships within the same table, like a manager and their employees, members of a family tree, or folders inside other folders.

PRIMARY KEY	FOREIGN KEY
A primary key is used to ensure data in the specific column is unique.	A foreign key is a column or group of columns in a relational database table that provides a link between data in two tables.
It uniquely identifies a record in the relational database table.	It refers to the field in a table which is the primary key of another table.
Only one primary key is allowed in a table.	Whereas more than one foreign key is allowed in a table.
It is a combination of UNIQUE and Not Null constraints.	It can contain duplicate values and a table in a relational database.
It does not allow NULL values .	It can also contain NULL values.
Its value cannot be deleted from the parent table.	Its value can be deleted from the child table.
It constraint can be implicitly defined on the temporary tables.	It constraint cannot be defined on the local or global temporary tables.

